

# Création application Web Asp.net Core, MVC, EFCore, Sql Server/Sqlite et déploiement sur Azure

Version 08.06.2026

## Table des matières

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| Création application Web Asp.net Core, MVC, EFCore, Sql Server/Sqlite et déploiement sur Azure .... | 1  |
| Versions .....                                                                                      | 2  |
| Création du projet ASP.NET Core MVC .....                                                           | 3  |
| Arborescence.....                                                                                   | 5  |
| Installation d'EntityFramework Core .....                                                           | 5  |
| Installation Docker pour Sql Server .....                                                           | 6  |
| Scaffolding avec EntityFramework Core .....                                                         | 6  |
| Annotations Entities .....                                                                          | 9  |
| Connexion à la base de données.....                                                                 | 9  |
| Création manuelle d'un CRUD.....                                                                    | 10 |
| Connexion à l'application avec utilisateur/mot de passe .....                                       | 16 |
| Infos de l'utilisateur dans les vues et controleurs.....                                            | 19 |
| Création d'un Controller avec Vues (Entity Framework) .....                                         | 21 |
| Création d'une combobox.....                                                                        | 23 |
| Page « facture » avec ajout ligne en JS et sauvegarde en JQUERY .....                               | 23 |
| Impression PDF.....                                                                                 | 28 |
| Création d'une API.....                                                                             | 32 |
| Serveur API .....                                                                                   | 32 |
| Client API .....                                                                                    | 32 |
| Custom SQL avec DbContext.....                                                                      | 33 |
| Créer un package pour nuget.....                                                                    | 33 |
| Copilot dans Visual Studio 2022.....                                                                | 34 |
| Déploiement de l'application sur Azure.....                                                         | 35 |
| Localisation.....                                                                                   | 38 |
| Problème certificat https.....                                                                      | 39 |
| TODO .....                                                                                          | 39 |

## Versions

08.06.2026 : ajout création package nuget

01.06.2026 : ajouter custom SQL

26.05.2026 : ajout commande docker + User.identity dans Controller

12.05.2026 : ajout résolution problème certificat https

11.05.2026 : modification table avec filtre

24.11.2025 : ajout numéro version pour install efcore

22.07.2025 : ajout connexion MySQL

16.06.2025 : ajout partie API

02.06.2025 : ajout déploiement azure

26.05.2025 : changement width pour document word, migration SQLITE – SQLSERVER, déploiement Azure

20.05.2025 : ajout User.Identity

19.05.2025 : ajout menu si admin ou non

28.04.2025 : ajout champ checkbox et url template table bootstrap

24.04.2025 : Ajout Docker

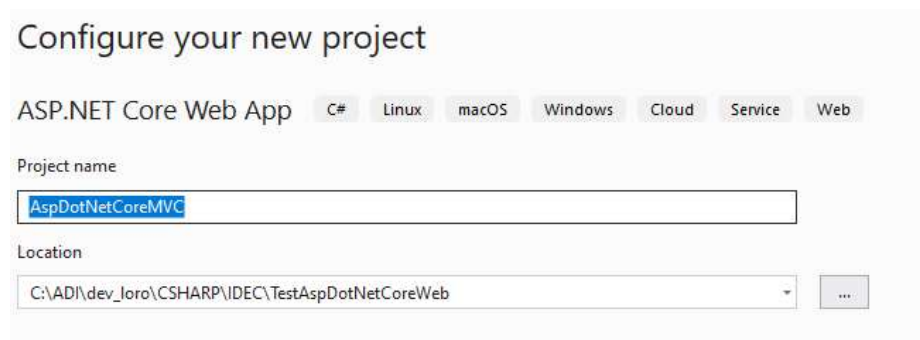
## Création du projet ASP.NET Core MVC

Dans Visual Studio 2022

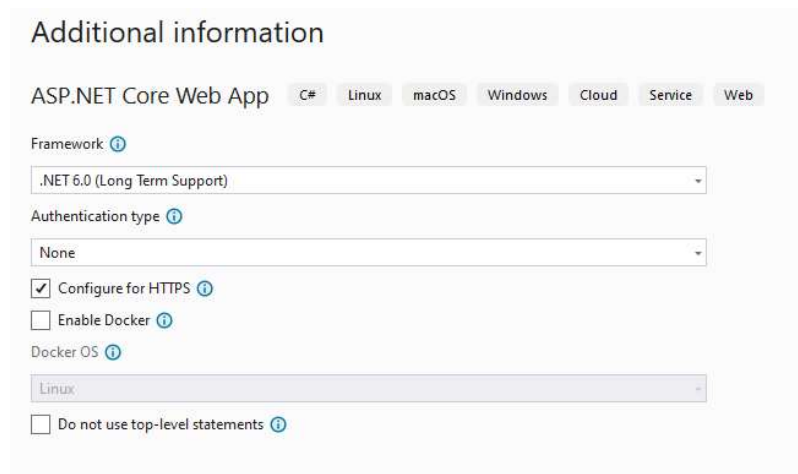
Ajouter un projet « ASP.NET Core Web App (Model-View-Controller) »

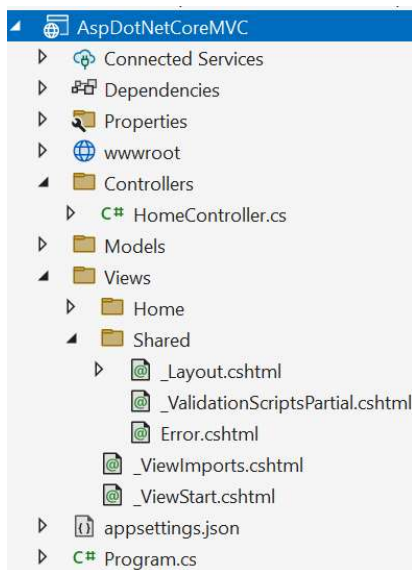


Nom du projet : AspDotNetCoreMVC



Laisser par défaut (sélectionner .NET 8.0 LTS)





Properties/launchSettings.json (applicationUrl http https)

wwwroot/css/site.css (faire un exemple)

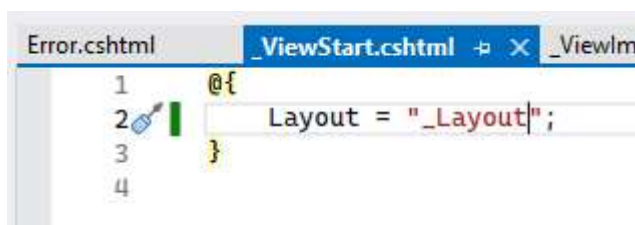
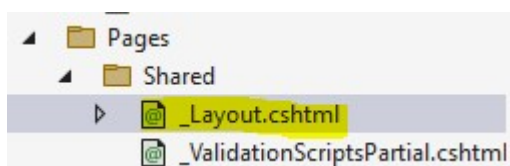
wwwroot/js/site.js (faire un exemple)

appsettings.json

Program.cs



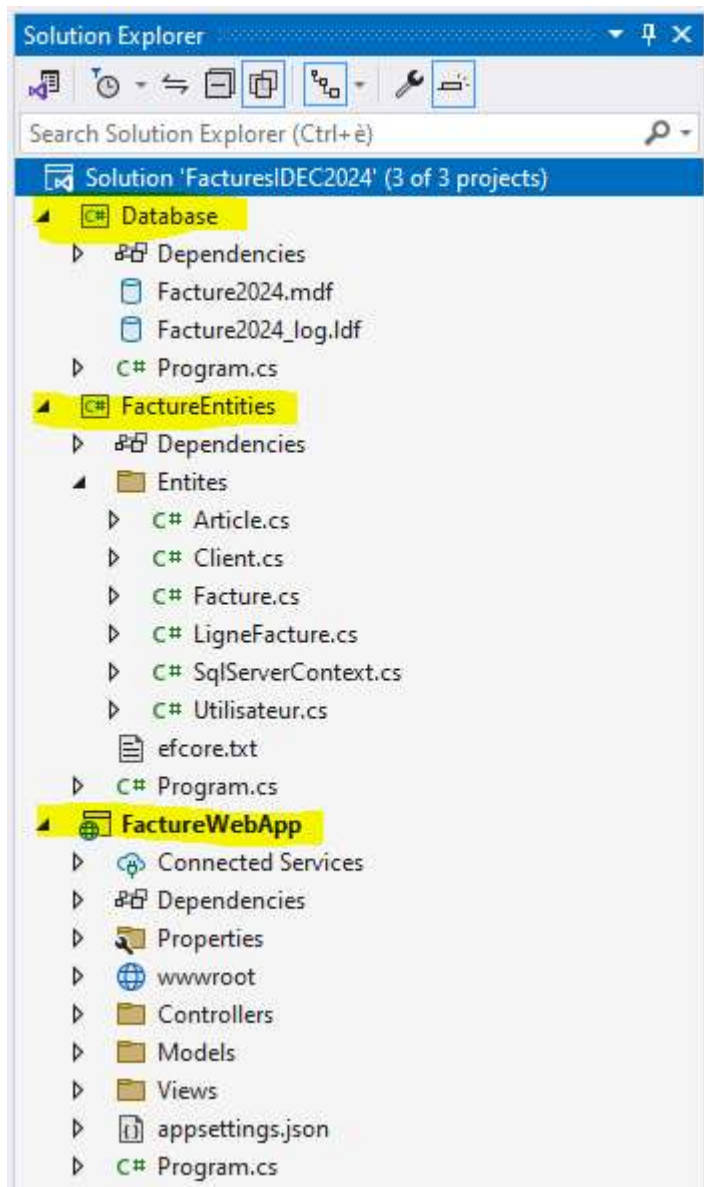
Le layout du site (menu, pied de page, référence au js, css, bootstrap, ...)



Explication Controller, Action, View, ... (Home, Index)

## Arborescence

Dans la solution, il faut créer le projet asp.net core, un projet console App (pour y mettre la base de données .mdf) et un projet console App pour y créer les entités. Voici un exemple :



## Installation d'EntityFramework Core

Pour connaître la version de .Net installée

```
dotnet --info
```

Pour installer la dernière version de .Net Core

```
dotnet tool install --global dotnet-ef  
dotnet tool install --global dotnet-ef --version 8.0.11
```

Pour mettre à jour la version

```
dotnet tool update --global dotnet-ef
```

Pour connaître la version de .Net Core installée

## Installation Docker pour Sql Server

Télécharger docker desktop : <https://www.docker.com/products/docker-desktop/>

Sur <http://mcr.microsoft.com/>, cliquer sur "Category - Databases" et sélectionner "Microsoft SQL Server Ubuntu based Images"

Dans "About" copier la commande pour 2019-latest

Lancer la commande dans un terminal : `docker pull mcr.microsoft.com/mssql/server:2019-latest`

Lancer la commande : `docker run -e "ACCEPT_EULA=Y" -e`

`"MSSQL_SA_PASSWORD=yourStrong(!)Password" -p 1433:1433 --name sqlserver --hostname sqlserver -d mcr.microsoft.com/mssql/server:2019-latest`

- ON TROUVE LA COMMANDE DANS LA PARTIE "USAGE" DE LA PAGE DE TELECHARGEMENT

- en lançant la commande cela créé le docker dans docker desktop aussi (avec le bon nom)

En passant la commande `docker ps` on doit voir apparaître le docker up

Ensuite on se connecte à la base de données :

- server : localhost

- login : sa

- password : yourStrong(!)Password

- select @@VERSION

`docker stop sqlserver`

`docker desktop start`

`docker container ls -a`

`docker start [name]`

Url utiles : <https://hub.docker.com/>

--> podman

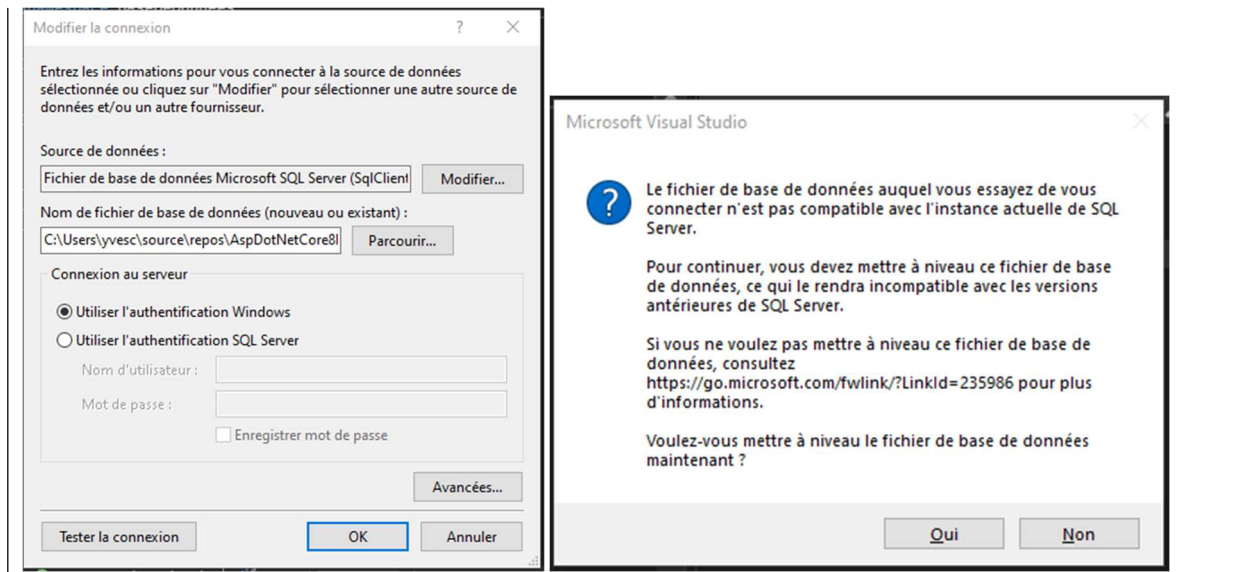
## Scaffolding avec EntityFramework Core

### [SQL SERVER]

Si double clic sur la base de données (.mdf) et que ce message apparaît

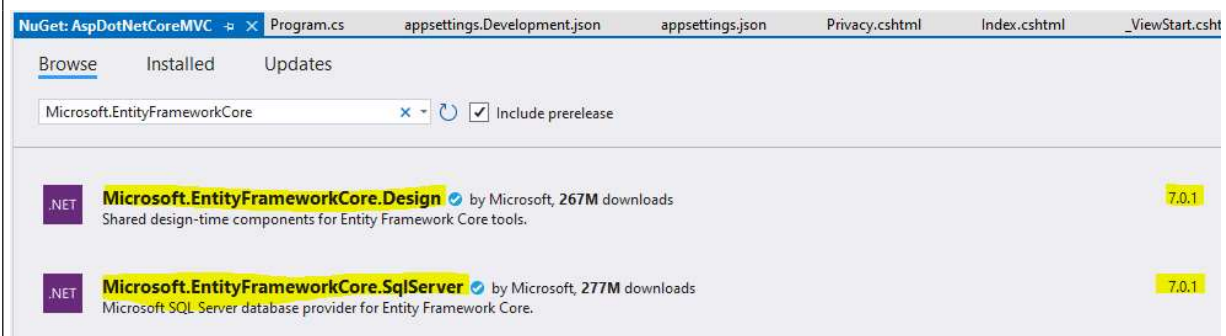


Dans l'explorateur de serveurs il faut faire un clic droit sur la base de données – « Modifier la connexion », puis cliquer directement sur « OK » et « Oui »



## [SQL SERVER]

Dans le projet Entities, installer 2 packages nuget afin de faire la connexion à la base de données Sql Server. Attention installer la dernière version de 8.x (car en .net core 8. Cela doit correspondre entre les versions utilisées)

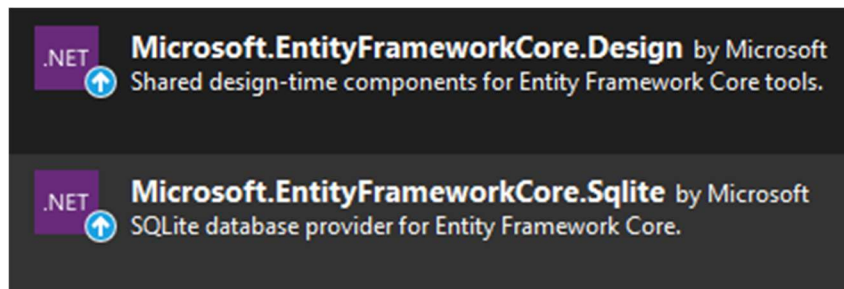


Build du projet puis scaffolding dans un powershell depuis VS :

```
dotnet ef dbcontext scaffold "Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\ADI\dev_loro\CSHARP\IDEC\TestAs
pDotNetCoreWeb\Facture.mdf;Integrated Security=True;"
"Microsoft.EntityFrameworkCore.SqlServer" -p AspDotNetCoreMVC -f -o Entites -c
SqlServerContext -d --no-onconfiguring
```

## [SQLITE]

Dans le projet Entities, installer 2 packages nuget afin de faire la connexion à la base de données Sqlite. Attention installer la dernière version de 8.x (car en .net core 8. Cela doit correspondre entre les versions utilisées)

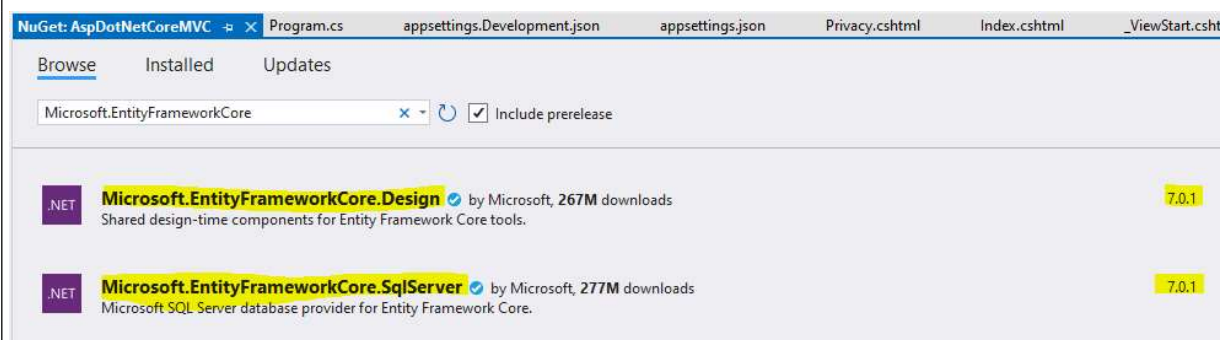


Build du projet puis scaffolding dans un powershell depuis VS :

```
dotnet ef dbcontext scaffold "Data Source=C:\ADI\dev_loro\CSHARP\GITHUB\blazor-
full-example\GestionArticles.Database\blazor-full-example.db"
"Microsoft.EntityFrameworkCore.Sqlite" -p GestionArticles.Entities -f -o Entites
-c SqliteContext -d -t Article -t Utilisateur
```

## [DOCKER - SQL SERVER]

Dans le projet Entities, installer 2 packages nuget afin de faire la connexion à la base de données Sql Server. Attention installer la dernière version de 8.x (car en .net core 8. Cela doit correspondre entre les versions utilisées)

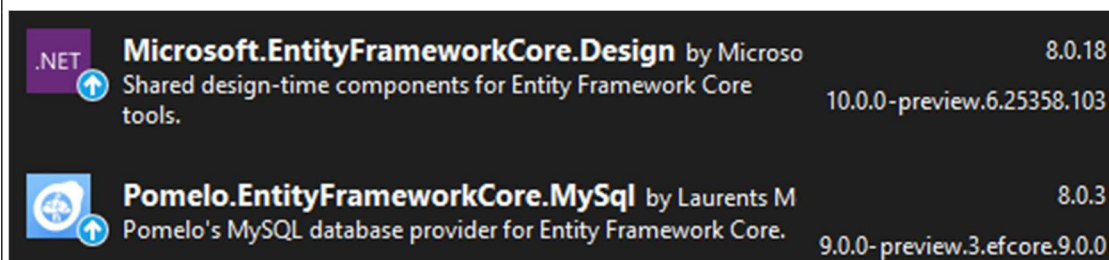


Build du projet puis scaffolding dans un powershell depuis VS :

```
dotnet ef dbcontext scaffold "Server=localhost,1433;user
id=sa;password=yourStrong(!)Password;database=test;TrustServerCertificate=True;"
"Microsoft.EntityFrameworkCore.SqlServer" -p efcoresqlserverlocaldocker -f -o
Entites -c SqlServerContext -d --no-onconfiguring
```

## MYSQL

Dans le projet Entities, installer 2 packages nuget afin de faire la connexion à la base de données MySQL. Attention installer la dernière version de 8.x (car en .net core 8. Cela doit correspondre entre les versions utilisées)



Build du projet puis scaffolding dans un powershell depuis VS :

```
dotnet ef dbcontext scaffold
"server=localhost;user=asdf;password=asdf;database=asdf"
"Pomelo.EntityFrameworkCore.MySql" -p ConsoleApp1 -f -o Entites -c MySQLContext -
d --no-onconfiguring
```

Créer un fichier texte et mettre cette commande dedans au cas où il faut l'exécuter à nouveau

## Annotations Entities

Pour les annotations sur les entités, voici des exemples d'utilisation :

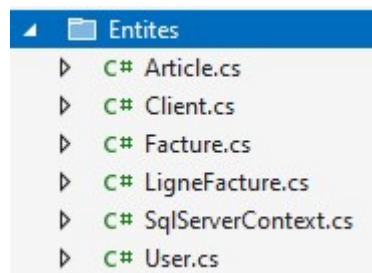
```
[Required]

[Required(ErrorMessage = "le nom de l'article est requis")]

[Range(0.05, 1000, ErrorMessage = "Le prix de l'article doit être en 0.05 CHF et
1000 CHF")]

[StringLength(8, ErrorMessage = "{0} la longueur doit être entre {2} et {1}.",
MinimumLength = 4)]

[DataType(DataType.Date)]//permet de ne pas mettre l'heure sinon
DataType.DateTime
[Display(Name = "Date de création")]//label
```



Url utile pour les différentes commandes : <https://learn.microsoft.com/en-us/ef/core/cli/dotnet>

## Connexion à la base de données

### [SQL SERVER]

Dans Program.cs ajouter les lignes suivantes pour ouvrir une connexion à la base de données

```
//connexion Sql Server
string sqlConnection =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<SqlServerContext>(options =>
options.UseSqlServer(sqlConnection));
```

dans le fichier appsettings.json, ajouter :

```
"ConnectionStrings": {
  "DefaultConnection": "Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=C:\\ADI\\dev_loro\\CSHARP\\IDEC\\
TestAspNetCoreWeb\\Facture.mdf;Integrated Security=True;"
}
```

Clean, build

### [SQLITE]

Dans Program.cs ajouter les lignes suivantes pour ouvrir une connexion à la base de données

```
//connexion Sql Server
string sqlConnection =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<SqliteContext>(options =>
options.UseSqlite(sqlConnection));
```

dans le fichier appsettings.json, ajouter :

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=C:\\ADI\\dev_lore\\CSHARP\\GITHUB\\blazor-full-
example\\GestionArticles.Database\\blazor-full-example.db"
}
```

Clean, build

Pour utiliser un .Include dans un Linq, il faut faire un using Microsoft.EntityFrameworkCore ;

## Création manuelle d'un CRUD

Préparer la classe « User » avec Metadata :

Il faut installer le package Microsoft.AspNetCore.Mvc.Core dans la solution

!!! les models partiels doivent être dans le même projet (donc Entities), pas possible de séparer dans 2 projets !!!

```
//Classe générée par EFCore
public partial class User
{
    public int Id { get; set; }
    public string Login { get; set; } = null!;
    public string Password { get; set; } = null!;
}
```

```
//2ème classe partielle dans le même projet
using Microsoft.AspNetCore.Mvc;
[ModelMetadataTypeAttribute(typeof(UserMetaData))]
public partial class User
{
}

public partial class UserMetaData
{
    [Required(ErrorMessage = "Le Login est obligatoire")]
    [MinLength(4, ErrorMessage = "minimum 4 caractères")]
    public string Login { get; set; } = null!;
    [Required(ErrorMessage = "Le Mot de passe est obligatoire")]
    [DisplayName("Mot de passe")]
    public string Password { get; set; } = null!;
}
```

Créer un controller empty

```
public class UsersController : Controller
```

```

{
    private readonly SqlServerContext _context;

    public UsersController(SqlServerContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        IEnumerable<User> users = _context.Users;
        return View(users);
    }

    public IActionResult Edit(int? id)
    {
        if (id == null || id == 0)
        {
            return NotFound();
        }

        User u = _context.Users.Find(id);
        if (u == null)
        {
            return NotFound();
        }
        ViewBag.valeur = "salut";
        return View(u);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Edit(User u, string valeur)
    {
        if (ModelState.IsValid)
        {
            _context.Users.Update(u);
            _context.SaveChanges();
            TempData["ok"] = "l'utilisateur a bien été mis à jour";
            return RedirectToAction("Index");
        }
        Console.WriteLine(Valeur);
        return View(u);
    }

    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(User u)
    {
        if (ModelState.IsValid)
        {
            _context.Users.Add(u);
            _context.SaveChanges();
            TempData["ok"] = "l'utilisateur a bien été créé";
            return RedirectToAction("Index");
        }
        return View(u);
    }
}

```

Exemple d'une page avec table, pagination, filtre, tri, ...

<https://codepen.io/delphives/pen/gxWMvN>

## Liste des articles

Ajouter un article

Rechercher...

| ID | Nom                | Prix     | Date d'achat | Utilisateur     | Actif                               |                                                      |
|----|--------------------|----------|--------------|-----------------|-------------------------------------|------------------------------------------------------|
| 1  | Jeu vidéo          | 41,75    | 20/03/2024   | Collet Yves     | <input checked="" type="checkbox"/> | <button>Modifier</button> <button>Supprimer</button> |
| 2  | Abonnement piscine | 360      | 01/01/2025   | Krenger Quentin | <input checked="" type="checkbox"/> | <button>Modifier</button> <button>Supprimer</button> |
| 3  | Ordinateur         | 899      | 04/09/2005   | Figueira Luis   | <input checked="" type="checkbox"/> | <button>Modifier</button> <button>Supprimer</button> |
| 4  | Voiture            | 10499,95 | 01/05/2006   | Figueira Luis   | <input type="checkbox"/>            | <button>Modifier</button> <button>Supprimer</button> |
| 5  | Logiciels          | 123,45   | 19/07/2026   | Collet Yves     | <input checked="" type="checkbox"/> | <button>Modifier</button> <button>Supprimer</button> |

1 à 5 entrée(s) sur 6 au total

Précédent 1 2 Suivant

Dans \_Layout.cshtml ajouter à la fin de head :

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

Et commenter la ligne à la fin :

```
<script src="~/lib/jquery/dist/jquery.min.js"></script>
```

Dans la page list.cshtml

```
<h1>Liste des articles</h1>
<div class="d-flex justify-content-end mb-3">
  <button class="btn btn-success btn-sm">Ajouter un article</button>
</div>
<style>
  /* Personnalisation des icônes de tri */
  th.sorting:after, th.sorting_asc:after, th.sorting_desc:after {
    font-family: "Font Awesome 5 Free";
    font-weight: 900;
    margin-left: 5px;
  }

  th.sorting:after {
    content: "\f0dc"; /* Icône tri non activé (barres) */
  }

  th.sorting_asc:after {
    content: "\f0de"; /* Flèche haut */
  }

  th.sorting_desc:after {
    content: "\f0dd"; /* Flèche bas */
  }

  /* Ajustement des largeurs de colonnes */
  th, td {
    white-space: nowrap;
  }

  th:nth-child(2), td:nth-child(2) {
    width: 100%;
  }

  th:nth-child(1), th:nth-child(3), th:nth-child(4), th:nth-child(5),
  th:nth-child(6), th:nth-child(7), th:nth-child(8),
```

```

        td:nth-child(1), td:nth-child(3), td:nth-child(4), td:nth-child(5),
        td:nth-child(6), td:nth-child(7), td:nth-child(8) {
            width: 1%;
        }

        /* Alignement à droite pour la 3ème colonne */
        th:nth-child(3), td:nth-child(3) {
            text-align: right;
        }

</style>

<p>
    <div class="container-fluid">
        <table id="tableArticles" class="table table-striped table-bordered
table-sm">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Nom</th>
                    <th>Prix</th>
                    <th>Date d'achat</th>
                    <th>Utilisateur</th>
                    <th>Actif</th>
                    <th></th>
                    <th></th>
                </tr>
            </thead>
            <tbody>
                @foreach (Article art in articles)
                {
                    <tr>
                        <td>@art.Id</td>
                        <td>@art.Nom</td>
                        <td>@art.Prix</td>
                        <td>@art.DateAchat</td>
                        <td>@art.IdUtilisateurNavigation?.NomPrenom ?? "-
")</td>
                        <td><input type="checkbox" checked="@art.EstActif"
readonly="readonly" /></td>
                        <td><button class="btn btn-primary btn-
sm">Modifier</button></td>
                        <td><button class="btn btn-danger btn-
sm">Supprimer</button></td>
                    </tr>
                }
            </tbody>
        </table>
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js
"></script>
    <script
src="https://cdn.datatables.net/1.13.4/js/jquery.dataTables.min.js"></script>
    <script
src="https://cdn.datatables.net/1.13.4/js/dataTables.bootstrap5.min.js"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.4/css/all.min.css">
    <script>
        $(document).ready(function () {
            var table = $('#tableArticles').DataTable({
                "paging": true,

```

```

        "lengthChange": false,
        "pageLength": 5,
        "ordering": true,
        "order": [],
        "columnDefs": [
            { "orderable": false, "targets": [0, 2, 4, 5, 6, 7] },
            { "orderable": true, "targets": [1, 3] }
        ],
        "language": {
            "search": "", // Champ de recherche personnalisé
            "paginate": {
                "previous": "Précédent",
                "next": "Suivant"
            },
            "info": "_START_ à _END_ entrée(s) sur _TOTAL_ au total",
            "infoEmpty": "Aucune entrée",
            "infoFiltered": "(filtré de _MAX_ entrées au total)",
            "zeroRecords": "Aucun enregistrement trouvé",
        },
        "dom": "<'row'<'col-md-6'l><'col-md-6 d-flex justify-content-
end mb-3'f>>" +
            "<'row'<'col-sm-12'tr>>" +
            "<'row'<'col-md-6'i><'col-md-6 d-flex justify-content-
end'p>>",
        "initComplete": function () {
            $(".dataTables_filter input")
                .attr("placeholder", "Rechercher...")
                .addClass("form-control");
        }
    });

    $('#search').on('keyup', function () {
        table.search(this.value).draw();
    });
}
</script>
</p>

```

Créer la page index.cshtml

```

@using AspDotNetCoreMVC.Entites
@model IEnumerable<User>
@{
    ViewData["Titre"] = "Index Utilisateurs";
}
<h1>@ViewData["Titre"]</h1>

@{
    if (TempData["ok"] != null)
    {
        <h2>@TempData["ok"]</h2>
    }
}
<a asp-controller="Users" asp-action="Create" class="btn btn-primary">Créer</a>
<table class="table table-bordered table-striped" style="width:100%">
    <thead>
        <tr>
            <th>Id</th>
            <th>Login</th>
            <th>Password</th>
            <th></th>
        </tr>
    </thead>

```

```

</thead>
<tbody>
  @foreach(User u in Model)
  {
    <tr>
      <td>@u.Id</td>
      <td width="50%">@u.Login</td>
      <td width="50%">@u.Password</td>
      <td><a asp-controller="Users" asp-action="Edit" asp-route-
id="@u.Id" class="btn btn-primary mx-2">Modifier</a></td>
    </tr>
  }
</tbody>
</table>

```

Créer la page "Create.cshtml »

```

@using AspDotNetCoreMVC.Entites
@model User;
<h2>Création d'un utilisateur</h2>
<form method="post" asp-action="Create">
  <div asp-validation-summary="All"></div>
  <label asp-for="Login"></label>
  <input asp-for="Login" class="form-control" />
  <span asp-validation-for="Login" class="text-danger"></span>
  <br/>
  <label asp-for="Password"></label>
  <input asp-for="Password" class="form-control" />
  <span asp-validation-for="Password" class="text-danger"></span>
  <br/>
  <button type="submit" class="btn btn-primary">Créer</button>
</form>

```

Copier la page « Create.cshtml » et la renommer en « Edit.cshtml »

```

@using AspDotNetCoreMVC.Entites
@model User;
<h2>Edition d'un utilisateur</h2>
<form method="post" asp-action="Edit">
  <div asp-validation-summary="All"></div>
  <label asp-for="Login"></label>
  <input asp-for="Login" class="form-control" />
  <span asp-validation-for="Login" class="text-danger"></span>
  <br/>
  <label asp-for="Password"></label>
  <input asp-for="Password" class="form-control" />
  <span asp-validation-for="Password" class="text-danger"></span>
  <br/>
  <label>Valeur</label>
  <input name="valeur" value="@ViewBag.valeur" class="form-control" />
  <br/>
  <label asp-for="Autorise"></label>
  <input asp-for="Autorise" type="checkbox" />
  <span asp-validation-for="Autorise" class="text-danger"></span>
  <br />
  <button type="submit" class="btn btn-primary">Sauver</button>
</form>

```

## Connexion à l'application avec utilisateur/mot de passe

Pour le chiffrement du mot de passe :

```
using System.Security.Cryptography;

namespace Helpers
{
    public class Security
    {
        private const int _saltSize = 16; // 128 bits
        private const int _keySize = 32; // 256 bits
        private const int _iterations = 50000;
        private static readonly HashAlgorithmName _algorithm =
        HashAlgorithmName.SHA256;

        private const char segmentDelimiter = ':';

        public static string Hash(string input)
        {
            byte[] salt = RandomNumberGenerator.GetBytes(_saltSize);
            byte[] hash = Rfc2898DeriveBytes.Pbkdf2(
                input,
                salt,
                _iterations,
                _algorithm,
                _keySize
            );
            return string.Join(
                segmentDelimiter,
                Convert.ToHexString(hash),
                Convert.ToHexString(salt),
                _iterations,
                _algorithm
            );
        }

        public static bool Verify(string input, string hashString)
        {
            string[] segments = hashString.Split(segmentDelimiter);
            byte[] hash = Convert.FromHexString(segments[0]);
            byte[] salt = Convert.FromHexString(segments[1]);
            int iterations = int.Parse(segments[2]);
            HashAlgorithmName algorithm = new HashAlgorithmName(segments[3]);
            byte[] inputHash = Rfc2898DeriveBytes.Pbkdf2(
                input,
                salt,
                iterations,
                algorithm,
                hash.Length
            );
            return CryptographicOperations.FixedTimeEquals(inputHash, hash);
        }
    }
}
```

Ajouter une page « Views/Login/Login.cshtml »

(demander à chatgpt)

<https://mdbootstrap.com/docs/standard/extended/login/>

@{

```

        Layout = null;
    }
    <!DOCTYPE html>
    <html lang="en">
    <head>
        <meta charset="utf-8" />
        <title>Connexion</title>
        <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
        <link rel="stylesheet" href="~/css/site.css" />
    </head>
    <body>
        <form method="post" asp-controller="Login" asp-action="Connexion">
            <section class="vh-100" style="background-color: #508bfc;">
                <div class="container py-5 h-100">
                    <div class="row d-flex justify-content-center align-items-center h-100">
                        <div class="col-12 col-md-8 col-lg-6 col-xl-5">
                            <div class="card shadow-2-strong" style="border-radius: 1rem;">
                                <div class="card-body p-5 text-center">
                                    <h3 class="mb-5">Connexion</h3>
                                    <div class="form-outline mb-4">
                                        <input type="text" name="utilisateur" class="form-control" placeholder="Utilisateur" required />
                                    </div>
                                    <div class="form-outline mb-4">
                                        <input type="password" name="motdepasse" class="form-control" placeholder="Mot de passe" required />
                                    </div>
                                    <button class="btn btn-primary" type="submit">Se
                                </div>
                                <partial name="_Notification" />
                            </div>
                        </div>
                    </div>
                </div>
            </section>
        </form>
    </body>
    </html>

```

Créer le controller « LoginController.cs »

```

public class LoginController : Controller
{
    private readonly SqlServerContext _context;
    public LoginController(SqlServerContext context)
    {
        _context = context;
    }
    public IActionResult Login()
    {
        if (HttpContext.User.Identity.IsAuthenticated)
            return RedirectToAction("Index", "Home");

        return View("Login");
    }
}

```

```

    }

    [HttpPost]
    public IActionResult Connexion(string utilisateur, string motdepasse)
    {
        if (HttpContext.User.Identity.IsAuthenticated)
            return Redirect("~/Home/Index");

        if (string.IsNullOrEmpty(utilisateur) ||
            string.IsNullOrEmpty(motdepasse))
        {
            TempData["ko"] = "Veuillez saisir un utilisateur et un mot de
            passe";
            return Redirect("~/Login/Login");
        }

        Utilisateur u = _context.Utilisateurs.FirstOrDefault(ux => ux.Login
        == utilisateur && ux.Autorise);
        if (u != null)
        {
            if (!Security.Verify(motdepasse, u.MotDePasse))
            {
                u = null;
            }
        }
    }

    if (u == null)
    {
        TempData["ko"] = "Echec de lors de la connexion";
        return Redirect("~/Login/Login");
    }

    string claimRole = u.Role;

    var userClaims = new[] {
        new Claim("Login", utilisateur),
        new Claim("Role", claimRole),
        new Claim(ClaimTypes.Name, utilisateur), //pour authorize
        new Claim(ClaimTypes.Role, claimRole), //pour authorize
        new Claim("Id", Convert.ToString(u.Id))
    };
    ClaimsIdentity claimsIdentity = new ClaimsIdentity(userClaims,
    "custom");
    //--> il faut laisser custom pour que HttpContext.User.IsAuthenticated soit à
    True

    ClaimsPrincipal userPrincipal = new ClaimsPrincipal(new[] {
    claimsIdentity });
    HttpContext.User = userPrincipal;
    HttpContext.SignInAsync(userPrincipal);

    HttpContext.Session.SetString("id", Convert.ToString(u.Id));
    HttpContext.Session.SetString("userName", utilisateur);
    HttpContext.Session.SetString("role", u.Role);
    return Redirect("~/Home/Index");
}

[Authorize]
public IActionResult Logout()
{
    HttpContext.SignOutAsync();

    return Redirect("~/Login/Login");
}

```

```
}  
}
```

## Infos de l'utilisateur dans les vues et controlleurs

Dans « \_Layout.cshtml » ajouter :

```
@inject Microsoft.AspNetCore.Http.IHttpContextAccessor HttpContextAccessor
```

```
@if (HttpContextAccessor.HttpContext.Session.GetString("role") == "admin")  
{  
    <li class="nav-item">  
        <a class="nav-link text-dark" asp-area="" asp-controller="Utilisateur"  
asp-action="Index">Utilisateur</a>  
    </li>  
}
```

```
<li class="nav-item">  
    <a class="nav-link text-dark" asp-area="" asp-  
controller="Login" asp-action="Logout">Se déconnecter (Bonjour  
@HttpContextAccessor.HttpContext.Session.GetString("userName"))</a>  
</li>
```

## OU ALORS PLUS SIMPLE

```
@if (User.Identity.IsAuthenticated)  
{  
    <p>Bonjour @User.Identity.Name</p>  
    @if (User.IsInRole("admin"))  
    {  
        <p>Bienvenue, administrateur !</p>  
    }  
    else  
    {  
        <p>Bienvenue, utilisateur</p>  
    }  
}
```

```
@(User.Claims.FirstOrDefault(x => x.Type == "Login").Value)
```

Et dans les controllers

```
if(User.Identity.IsAuthenticated)  
{  
    Console.WriteLine(User.Identity.Name);  
    if(User.IsInRole("admin"))  
    {  
        Console.WriteLine(User.FindFirst("id") ?.Value);  
    }  
    else  
    {  
        Console.WriteLine("user");  
    }  
}
```

Dans Program.cs ajouter :

```
builder.Services.AddSession();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(options =>
{
    options.ReturnUrlParameter = CookieAuthenticationDefaults.ReturnUrlParameter;
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = Microsoft.AspNetCore.Http.CookieSecurePolicy.SameAsRequest;
    options.Cookie.Name = "UserLoginCookie";
    options.ExpireTimeSpan = new TimeSpan(0, 0, 60, 0);
    options.LoginPath = "/Login/Login";
    options.SlidingExpiration = true;
    options.Events = new CookieAuthenticationEvents();
});
builder.Services.AddHttpContextAccessor();
```

```
builder.Services.AddSession();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(options =>
{
    options.LoginPath = "/Login/Login";
    options.AccessDeniedPath = "/Home/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
});
builder.Services.AddHttpContextAccessor();
```

```
//!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!HYPER IMPORTANT QUE CETTE LIGNE SOIT AVANT
app.UseAuthentication();
app.UseAuthorization();
app.UseSession();
```

créer le fichier « \_Notification.cshtml » dans « Shared » :

```
@{
    if (TempData["ok"] != null)
    {
        <h2>@TempData["ok"]</h2>
    }
    if (TempData["ko"] != null)
    {
        <h2>@TempData["ko"]</h2>
    }
}
```

puis dans le layout :

```
<partial name="_Notification" />
```

Puis dans chaque Classe ou méthode mettre [Authorize]

```
[Authorize(Roles = "user")]
[Authorize(Roles = "user, admin")]
```

Créer la méthode pour rediriger si pas l'autorisation :

```
public class HomeController : Controller
{
    public IActionResult AccessDenied(string returnUrl)
    {
        TempData["ko"] = "Vous n'avez pas le droit d'accéder à cette page";
        return Redirect("~/Home/Index");
    }
}
```

## Création d'un Controller avec Vues (Entity Framework)

Si pendant la génération du scaffolding on tombe sur cette erreur

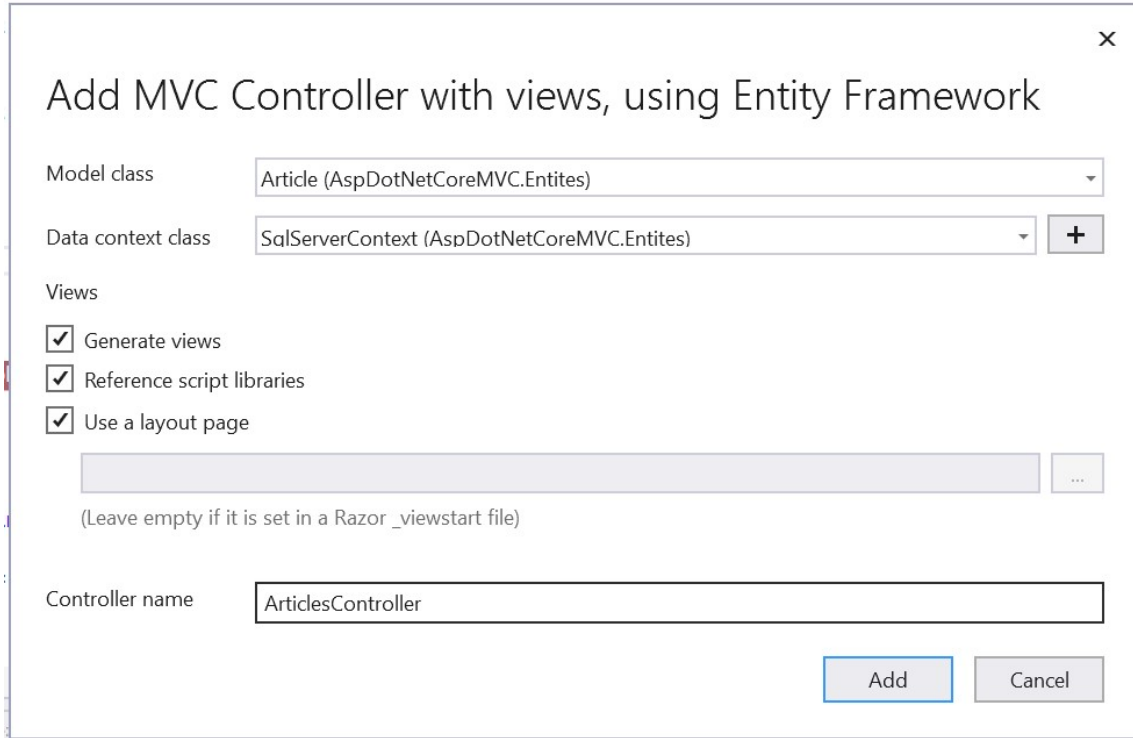


- !!! si on a pas le « onconfiguring » dans le scaffolding il faut instancier le dbcontext dans program.cs avec le builder sinon il ne sais pas comment se connecter et à quelle base !!!

Autres options :

- Fermer visual studio et rouvrir
- Recompiler la solution en entier
- Installer le package « Microsoft.VisualStudio.Web.CodeGeneration »
- Mettre à jour tous les packages de nuget

Ajouter un nouveau « Scaffolded Item » - « MVC Controller with views, using Entity Framework »



×

## Add MVC Controller with views, using Entity Framework

Model class:

Data context class:

Views

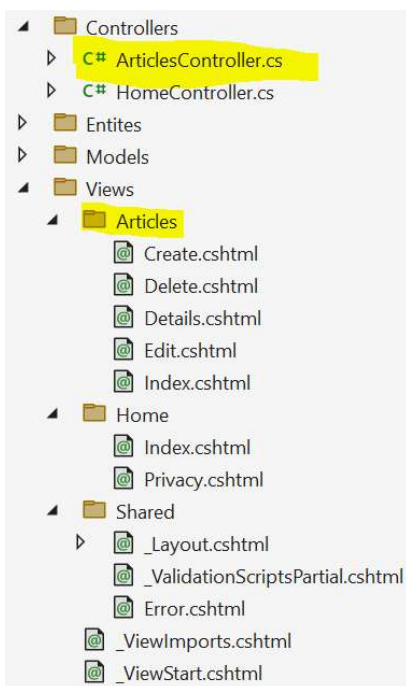
☒ Generate views

☒ Reference script libraries

☒ Use a layout page

(Leave empty if it is set in a Razor \_viewstart file)

Controller name:



Injection de dépendance avec le dbcontext en paramètre d'entrée du constructeur du controller

```

public class ArticlesController : Controller
{
    private readonly SqlServerContext _context;

    0 references
    public ArticlesController(SqlServerContext context)
    {
        _context = context;
    }
}

```

Explication return View avec ou sans paramètres. Le concept nom de controller, nom d'action et paramètre.

## Création d'une combobox

Dans le controller

```

public IActionResult Create()
{
    LoadListCategorie();
    //TODO à créer un article en entrée pour avoir les valeurs par défaut
    Article article = new Article();
    return View(article);
}

private void LoadListCategorie()
{
    List<SelectListItem> list = new List<SelectListItem>() {
        new SelectListItem()
        {
            Text = "[Sans catégorie]",
            Value = ""
        },
        new SelectListItem()
        {
            Text = "Sport",
            Value = "Sport"
        },
        new SelectListItem()
        {
            Text = "Informatique",
            Value = "Informatique"
        }
    };
    ViewBag.ListCategorie = list;
}

```

Dans la page html

```

<select asp-for="Categorie" class="form-control" asp-items="@ViewBag.ListCategorie"></select>

```

## Page « facture » avec ajout ligne en JS et sauvegarde en JQUERY

```

public class FactureController : RootController
{
    public FactureController(SqlServerContext context)
    {
        _context = context;
    }
}

```

```

    public async Task<IActionResult> Index()
    {
        string role = User.Claims.FirstOrDefault(x => x.Type == "Role").Value;
        int id = int.Parse(User.Claims.FirstOrDefault(x => x.Type ==
        "Id").Value);
        var maliste = _context.Factures.Include(f => f.Client).Include(f =>
        f.Details).ToList();
        return View(maliste);
    }

    public bool CreateNewFacture(List<Detail> LignesFacture, DateTime
    DateFacture, int ClientId, string NumeroFacture)
    {
        Facture facture = new Facture();
        facture.DateFacture = DateFacture;
        facture.ClientId = ClientId;
        facture.Details = LignesFacture;
        facture.NumeroFacture = NumeroFacture;
        _context.Add(facture);
        _context.SaveChanges();
        return true;
    }

    public IActionResult Create()
    {
        //création de la liste des clients
        List<SelectListItem> listClients = new List<SelectListItem>();
        foreach (Client c in _context.Clients)
        {
            listClients.Add(new SelectListItem() { Text = c.Prenom + " " + c.Nom,
            Value = c.Id.ToString() });
        }
        ViewData["Clients"] = listClients;
        //transformation de la liste des articles en json pour javascript plus
        tard
        ViewBag.ArticlesJson = JsonConvert.SerializeObject(_context.Articles);
        return View(new Facture() { DateFacture = DateTime.Now });
    }
}

```

La page Index.cshtml

```

@using Newtonsoft.Json;
@model List<Facture>

<h1>Liste des factures</h1>
<partial name="Message" />
<br />
<a asp-controller="Facture" asp-action="Create" class="btn btn-primary">Créer une
nouvelle Facture</a>
<table class="table table-striped table-bordered">
    <thead>
        <tr>
            <th>N°</th>
            <th>Date</th>
            <th>Client</th>
            <th>Nombre d'articles</th>
            <th>Total</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @{

```

```

        foreach (Facture f in Model)
        {
            <tr>
                <td width="5%">@f.Id</td>
                <td width="5%">@f.DateFacture</td>
                <td width="5%">@f.Client.Prenom</td>
                <td width="5%">@f.Details.Sum(x => x.Quantite)</td>
                <td width="5%">??</td>
                <td width="5%"><a asp-controller="Print" asp-action="Print"
asp-route-factureId="@f.Id" target="_blank" class="btn btn-primary mx-
2">Imprimer</a></td>
            </tr>
        }
    }
</tbody>
</table>

```

La page Create.cshtml

```

@using Newtonsoft.Json;
@model Facture

@{
    ViewData["Title"] = "Create";
}
<h4>Création d'une facture</h4>
<hr />
<div class="container">
    @*pour aligner à gauche (pas que ça soit réparti sur toute la largeur*@
    <div class="d-inline-flex align-items-start">
        <div asp-validation-summary="ModelOnly" class="text-danger"></div>
        @* me-5 pour laisser un margin entre les composants*@
        <div class="col-sm me-5">
            <div class="form-group">
                <label asp-for="NumeroFacture" class="control-label"></label>
                <input asp-for="NumeroFacture" class="form-control widthAuto" />
                <span asp-validation-for="NumeroFacture" class="text-
danger"></span>
            </div>
        </div>
        <br />
        <div class="col-sm me-5">
            <div class="form-group">
                <label asp-for="DateFacture" class="control-label"></label>
                <input asp-for="DateFacture" class="form-control widthAuto" />
                <span asp-validation-for="DateFacture" class="text-
danger"></span>
            </div>
        </div>
        <div class="col-sm">
            <div class="form-group">
                <label asp-for="ClientId" class="control-label"></label>
                <select asp-for="ClientId" class="form-control widthAuto" asp-
items="ViewBag.Clients">
                    <option value="" selected="selected">[Veuillez sélectionner
un client]</option>
                </select>
                <span asp-validation-for="ClientId" class="text-danger"></span>
            </div>
        </div>
    </div>
    <br />
    <div class="d-inline-flex align-items-start mt-4">

```

```

<div class="col-sm me-5">
  <div class="form-group">
    <select id="Articles"
            name="Articles"
            class="form-control widthAuto"
            size="10">
      @{
        List<Article> listArticles =
JsonConvert.DeserializeObject<List<Article>>(ViewBag.ArticlesJson);
      }
      @foreach (Article a in listArticles)
      {
        <option value="@a.Id">@a.Nom</option>
      }
    </select>
  </div>
</div>
<div class="col-sm">
  <div class="form-group">
    <input type="hidden" id="IdArticleToAdd" value="" />
    N° Article :
    <input type="text" readonly="readonly" id="Num" value=""
class="form-control widthAuto" />

    Nom :
    <input type="text" readonly="readonly" id="Nom" value=""
class="form-control widthAuto" />

    Quantité :
    <input type="number" min="1" id="Quantite" value="" class="form-
control widthAuto" />

    Prix unitaire :
    <input type="number" min="1" id="PrixUnitaire" value=""
class="form-control widthAuto" />
    <br />
    <!--TODO on est dans un form, pour les boutons qui appelle du
javascript il ne faut pas leur donner d'id ou name-->
    <input type="button" class="btn btn-primary" value="Ajouter"
onclick="AddArticle()" />
  </div>
</div>
</div>
<div class="row">
  <div class="col-md-4 mt-3">

    <table class="table table-striped table-sm" id="TableLignesFacture">
      <thead>
        <tr>
          <th scope="col">#</th>
          <th scope="col">Id</th>
          <th scope="col">Nom</th>
          <th scope="col">Quantité</th>
          <th scope="col">Prix</th>
          <th scope="col">Total</th>
          <th scope="col">&nbsp;</th>
        </tr>
      </thead>
      <tbody>
        @foreach (Detail d in Model.Details)
        {
          <tr>

```

```

        <th scope="row">@d.Id</th>
        <td>@d.Article.Id</td>
        <td>@d.Article.Nom</td>
        <td>@d.Quantite</td>
        <td>@d.PrixVente</td>
        <td>??</td>
        <td>delete</td>
    </tr>
    }
</tbody>
</table>
<div class="form-group">
    @*<input type="submit" value="Enregistrer" class="btn btn-primary"
/*>
    <input type="button" value="Sauver" id="Update"
onclick="GetLignesFacture()" />
</div>

</div>
</div>
@*</form>*@
<div>
    <a asp-action="Index">Retour à la liste</a>
</div>

<script>
    //quand on clique sur un article on affiche les infos dans la partie détail à
droite
    const select = document.getElementById('Articles');
    var id = 0;
    var nom = "";
    select.addEventListener('change', function handleChange(event) {
        id = event.target.value;
        //déserialisation de la liste
        var Articles =
JSON.parse(@Html.Raw(Json.Serialize(@ViewBag.ArticlesJson)));
        for (var i = 0; i < Articles.length; i++) {
            if (id == Articles[i].Id) {
                document.getElementById('IdArticleToAdd').value = Articles[i].Id;
                document.getElementById('Num').value = Articles[i].Id;
                document.getElementById('Nom').value = Articles[i].Nom;
                document.getElementById('Quantite').value = 1;
                document.getElementById('PrixUnitaire').value = Articles[i].Prix;
            }
        }
    });
</script>
<script>
    //ajout d'un article dans le tableau
    function AddArticle() {
        var id = $("#IdArticleToAdd").val();
        var nom = $("#Nom").val();
        var quantite = $("#Quantite").val();
        var prixUnitaire = $("#PrixUnitaire").val();
        if (id == '' || nom == '' || quantite == 0 || quantite == '' ||
prixUnitaire == 0 || prixUnitaire == '') {
            alert(id + '/' + nom + '/' + quantite + '/' + prixUnitaire);
        } else {
            var tr = "<tr><td>-</td><td>" + id + "</td><td>" + nom + "</td><td>"
+ quantite + "</td><td>" + prixUnitaire + "</td><td>" + (quantite * prixUnitaire)
+ "</td><td>X</td></tr>";
            $("#TableLignesFacture").append(tr);
        }
    }

```

```

    }
</script>
<script>
    //récupération des lignes dans la table pour les ajouter à lignes factures
    function GetLignesFacture() {
        var result = [];
        var tr = $("#TableLignesFacture tr");
        for (var i = 1; i < tr.length; i++) {
            var tds = $(tr[i]).find("td");
            if (tds.length > 0) {
                result.push({
                    "ArticleId": tds[1].innerHTML,
                    "Quantite": tds[3].innerHTML,
                    "PrixUnitaire": tds[4].innerHTML
                })
            }
        }
        //récupération des données de la nouvelle facture
        var date = document.getElementById('DateFacture').value;
        var client = document.getElementById('ClientId').value;
        var num = document.getElementById('NumeroFacture').value;
        var jsonData = {
            "LignesFacture": result,
            "DateFacture": date,
            "ClientId": client,
            "NumeroFacture": num
        }

        $.ajax({
            dataType: 'json',
            type: "Post",
            data: jsonData,
            url: "/Facture/CreateNewFacture",
            success: function (html) {
                // recharge la page pour afficher les factures
                window.location.replace("/Facture/Index");
            }
        })
    }
</script>
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

```

## Impression PDF

### Installer le package nuget « FreeSpire.Doc »

Dans le controller :

```

public IActionResult Impression(int factureId)
{
    Facture f = _monContext.Factures.Include(x =>
x.LigneFactures).ThenInclude(x => x.Article).Include(x =>
x.Client).FirstOrDefault(x => x.Id == factureId);
    string physicalPath = Print.CreateDocumentFromTemplateWithFormat(f,
@"C:\Users\yvco1\Desktop\facture_template.docx");
    byte[] pdfBytes = System.IO.File.ReadAllBytes(physicalPath);
}

```

```

        MemoryStream ms = new MemoryStream(pdfBytes);
        return new FileStreamResult(ms, "application/pdf");
    }

```

Dans le dossier Helpers, ajouter la classe :

```

public static string CreateDocumentFromTemplateWithFormat(Facture f, string
template)
{
    Document document = new Document();
    document.LoadFromFile(template);
    document.Replace("client_prenom", f.Client.Prenom, true, true);
    document.Replace("client_nom", f.Client.Nom, true, true);
    document.Replace("client_adresse", f.Client.Rue, true, true);
    document.Replace("client_npa", f.Client.Npa.ToString(), true, true);
    document.Replace("client_localite", f.Client.Localite, true, true);
    document.Replace("facture_date",
f.DateFacture.ToString("dd.MM.yyyy"), true, true);
    document.Replace("facture_num", f.Numero, true, true);

    //la première section est celle où l'on trouve un titre "Titre1"
    Section s = document.Sections[0];
    Table table = s.AddTable(true);
    String[] Header = { "N°", "Article", "Quantité", "Prix unitaire",
"Total" };
    table.ResetCells(f.LigneFactures.Count + 1, Header.Length);

    //Header Row
    TableRow FRow = table.Rows[0];
    FRow.IsHeader = true;
    //Row Height
    //FRow.Height = 18;
    FRow.Cells[0].SetCellWidth(20, CellWidthType.Point);
    FRow.Cells[1].SetCellWidth(150, CellWidthType.Point);
    FRow.Cells[2].SetCellWidth(60, CellWidthType.Point);
    FRow.Cells[3].SetCellWidth(80, CellWidthType.Point);
    FRow.Cells[4].SetCellWidth(40, CellWidthType.Point);
    //Header Format
    FRow.RowFormat.BackgroundColor = Color.LightBlue;
    for (int i = 0; i < Header.Length; i++)
    {
        //Cell Alignment
        Paragraph p = FRow.Cells[i].AddParagraph();
        FRow.Cells[i].CellFormat.VerticalAlignment =
VerticalAlignment.Middle;
        p.Format.HorizontalAlignment = HorizontalAlignment.Center;
        //Data Format
        TextRange TR = p.AppendText(Header[i]);
        TR.CharacterFormat.FontName = "Calibri";
        TR.CharacterFormat.FontSize = 14;
        TR.CharacterFormat.TextColor = Color.Teal;
        TR.CharacterFormat.Bold = true;
    }

    decimal grandTotal = 0;
    List<LigneFacture> listLignesFactures = new
List<LigneFacture>(f.LigneFactures);
    //Data Row
    for (int r = 0; r < listLignesFactures.Count; r++)
    {
        LigneFacture lf = listLignesFactures[r];
        TableRow DataRow = table.Rows[r + 1];
    }
}

```

```

        //Row Height
        DataRow.Height = 15;

        //C Represents Column. 5 -> nombre de colonnes
        for (int c = 0; c < 5; c++)
        {
            //Cell Alignment
            DataRow.Cells[c].CellFormat.VerticalAlignment =
VerticalAlignment.Middle;
            //Fill Data in Rows
            Paragraph p2 = DataRow.Cells[c].AddParagraph();
            TextRange TR2 = null;
            decimal total = lf.Montant * lf.Quantite;
            switch (c)
            {
                case 0:
                    TR2 = p2.AppendText(lf.Article.Id.ToString());
                    DataRow.Cells[c].SetCellWidth(20, CellWidthType.Point);
                    break;
                case 1:
                    TR2 = p2.AppendText(lf.Article.Nom);
                    DataRow.Cells[c].SetCellWidth(150, CellWidthType.Point);
                    break;
                case 2:
                    TR2 = p2.AppendText(lf.Quantite.ToString());
                    DataRow.Cells[c].SetCellWidth(60, CellWidthType.Point);
                    break;
                case 3:
                    TR2 = p2.AppendText(lf.Montant.ToString());
                    DataRow.Cells[c].SetCellWidth(80, CellWidthType.Point);
                    break;
                case 4:
                    TR2 = p2.AppendText(total.ToString());
                    DataRow.Cells[c].SetCellWidth(40, CellWidthType.Point);
                    grandTotal += total;
                    break;
                default:
                    Console.WriteLine("Erreur dans le numéro de
colonne");
                    break;
            }

            //Format Cells
            p2.Format.HorizontalAlignment = HorizontalAlignment.Center;
            TR2.CharacterFormat.FontName = "Calibri";
            TR2.CharacterFormat.FontSize = 12;
            TR2.CharacterFormat.TextColor = Color.Brown;
        }
    }

    //TOTAL
    Paragraph pa = s.AddParagraph();
    pa.AppendText("\n");
    TextRange t = pa.AppendText($"TOTAL : {f.MontantTotalCHF}");
    pa.Format.HorizontalAlignment = HorizontalAlignment.Right;
    t.CharacterFormat.FontName = "Calibri";
    t.CharacterFormat.FontSize = 16;
    t.CharacterFormat.TextColor = Color.SteelBlue;
    string save = @"c:\adi\xxx.pdf";
    document.SaveToFile(save, FileFormat.PDF);
    return save;

```

```
//ok pour .net framework
//System.Diagnostics.Process.Start("xxx.pdf");

//ok pour .net core
//var pr = new Process();
//pr.StartInfo = new ProcessStartInfo(@"xxx.pdf")
//{
//    UseShellExecute = true
//};
//pr.Start();

//document.SaveToFile("SpireDocTestModified.docx",
FileFormat.Docx2013);
}
```

## Création d'une API

### Serveur API

Dans le projet Web, créer un dossier « API » et un controller – API vide.

```
[Route("api/[controller]")]
[ApiController]
public class ArticleApiController : ControllerBase
{
    private readonly SqliteContext _context;
    public ArticleApiController(SqliteContext context)
    {
        _context = context;
    }

    [HttpGet("getArticleById")]
    public IActionResult GetArticle(int id)
    {
        Article a = _context.Articles.Include(ac =>
ac.Categorie).FirstOrDefault(ax => ax.NumArticle == id);
        if (a == null)
        {
            return NotFound();
        }
        return Ok(new
        {
            Id = a.NumArticle,
            Nom = a.Nom,
            Prix = a.Prix,
            Catgorie = a.Categorie.Nom,
            EstActif = a.EstActif
        });
    }
}
```

Pour appeler cette méthode depuis Postman p.ex :

<https://localhost:7280/api/ArticleAPI/getArticleById?id=2>

### Client API

```
internal class Program
{
    static void Main(string[] args)
    {
        string url = "https://localhost:7280/api/ArticleAPI/getArticleById?id=2";

        HttpClient client = new HttpClient();

        string response = client.GetStringAsync(url).Result;

        var data = JsonConvert.DeserializeObject<dynamic>(response);
        var data2 = JsonConvert.DeserializeObject<Article>(response);

        Console.WriteLine(data.nom);
        Console.WriteLine(data2.Nom);
        Console.ReadLine();
    }
}

public class Article
{
}
```

```

    public int NumArticle { get; set; }
    public string Nom { get; set; } = null!;
    public decimal Prix { get; set; }
    public string Categorie { get; set; }
    public bool EstActif { get; set; }
}

```

## Custom SQL avec DbContext

S'il faut exécuter du SQL standard à partir du DbContext :

```
await _context.Database.ExecuteSqlRawAsync("DELETE FROM Coucou WHERE Id = {0}", id);
```

## Créer un package pour nuget

- créer un projet class Library (Factures2026Security)

- créer un fichier README.md à la racine

```

# MonPackage

Une bibliothèque .NET 10 d'exemple.

## Installation

```bash
dotnet add package MonEntreprise.MonPackage

```

- sur le projet - package --> id de package (Idec.Facture2026Security), version du package (1.0.0), auteur (Yves), description (Sécurité du mot de passe), readme (pointer vers le fichier)

ou bien :

```

<PackageId>Idec.Facture2026Security</PackageId>
<Version>1.0.0</Version>
<Authors>Yves Collet</Authors>
<Description>Sécurité du mot de passe</Description>

```

--> attention le packageid doit être unique sur nuget.org. il faut attendre un moment pour que le package soit listé sur le site

- dans le projet

```
dotnet pack -c Release
```

(ça crée un package nupkg dans le répertoire Release)

- sur nuget.org créer une clé api (sur le profil, "api keys")

```
oy2er2xxxxhroxlv4
```

```
dotnet nuget push bin/Release/Idec.Facture2026Security.1.0.0.nupkg --api-key oy2er2gqyxxxxxxxxlv4
--source https://api.nuget.org/v3/index.json
```

- à chaque fois qu'on veut redéployer une version il faut incrémenter version. On reçoit un email quand s'est publié sur nuget.org

## Copilot dans Visual Studio 2022

pour les version récentes de Visual Studio il n'y a plus le petit icône en bas à gauche. du coup en haut à droite - settings - options - "Show inline complétions"

- tab pour accepter la suggestions
  - faire un commentaire en expliquant ce que l'on veut et retour à la ligne pour avoir une suggestion
  - ALT et . pour changer une suggestion
  - ALT et / pour avoir la fenêtre copilot qui s'ouvre dans le code
  - dans la fenêtre on peut taper / pour avoir des options
  - on sélectionne un bout de code, on fait ALT et /, et on écrit /OPTIMIZE et la touche ENTER
  - dans la fenêtre faire # et le nom\_d\_un\_fichier et la question (p.ex peux-tu me décrire les méthodes de ce fichier)
- ex : # Program.cs peux-tu me donner le nom des variables globales
- dans le chat p.ex on peut lui dire #le\_nom\_du\_fichier sur lequel on veut agir
  - describe the file #Program.cs
  - / et quelque chose propose plein de trucs
  - /askvs ?????

# Déploiement de l'application sur Azure

Sur Azure créer un « resource group » (factures2025)

Créer une « SQL databases »

sql server (factures2025.database.windows.net) login : yves, password : ldec2025

Service tier : Basic (5.39 USD par mois)

public endpoint, allow and add : Yes

attendre 2 minutes

## Create SQL Database

Microsoft

BasicsNetworkingSecurityAdditional settingsTagsReview + create

### Product details

SQL database  
by Microsoft  
[Terms of use](#) | [Privacy policy](#)

### Estimated cost per month

5.39 USD

### Terms

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

### Basics

|                           |                                  |
|---------------------------|----------------------------------|
| Subscription              | LoroG - DevQA                    |
| Resource group            | factures2025                     |
| Region                    | Switzerland North                |
| Database name             | factures2025                     |
| Server                    | (new) factures2025               |
| Authentication method     | SQL authentication               |
| Server admin login        | yves                             |
| Compute + storage         | Basic: 2 GB storage              |
| Backup storage redundancy | Locally-redundant backup storage |

### Networking

|                                                          |         |
|----------------------------------------------------------|---------|
| Allow Azure services and resources to access this server | Yes     |
| Add current client IP address<br>91.195.188.10           | Yes     |
| Private endpoint                                         | None    |
| Minimum TLS version                                      | 1.2     |
| Connection Policy                                        | Default |

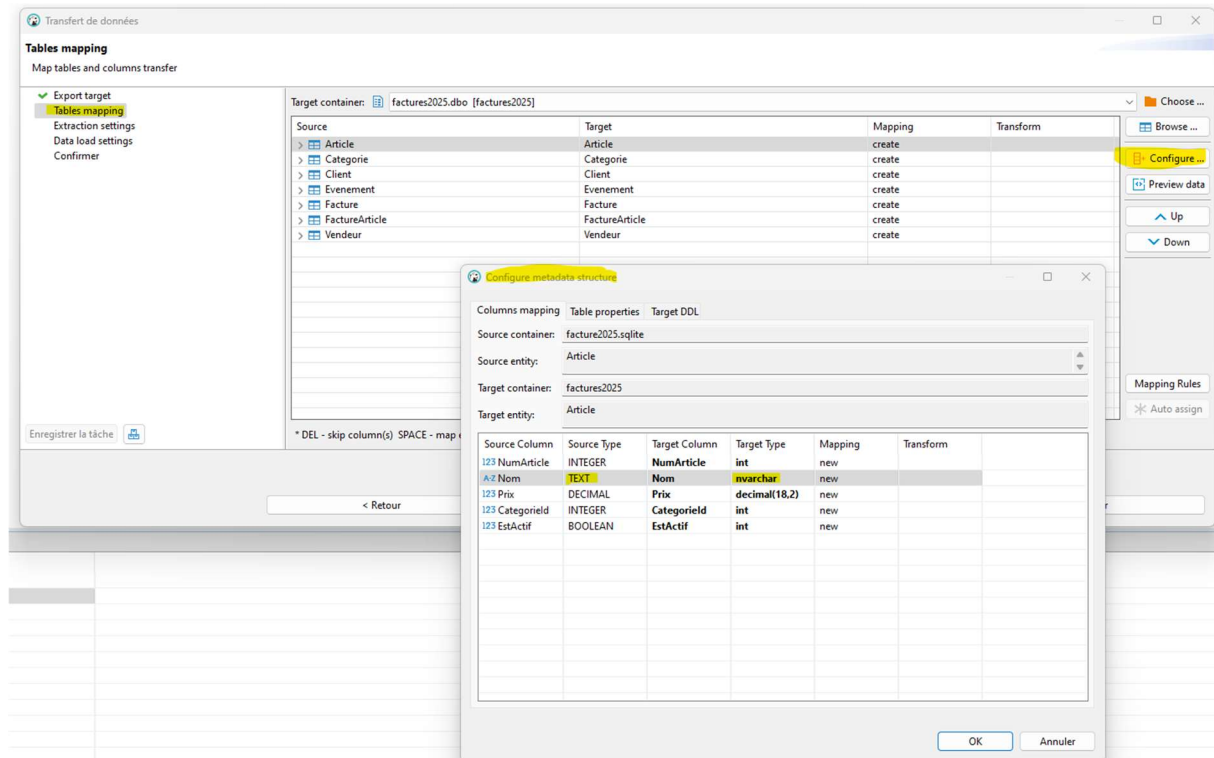
### Security

|                                               |                              |
|-----------------------------------------------|------------------------------|
| Identity                                      | Not enabled                  |
| Transparent data encryption (Server level)    | Service-managed key selected |
| Database level customer-managed key           | Not configured               |
| Database level user assigned managed identity | Not configured               |
| Advanced data security                        | Not now                      |
| Always Encrypted with secure enclaves         | Not configured               |

faire la migration depuis Sqlite vers la nouvelle SqlServer avec Dbeaver

clic sur les tables « Exporter les données »

ATTENTION : lors de l'export il faut changer les types TEXT en NVARCHAR et les DATEONLY en DATETIME et les BOOLEAN en BIT



nouvelle connectionstring :

```
Server=tcp:factures2025.database.windows.net,1433;Initial Catalog=factures2025;Persist Security Info=False;User ID=yves;Password=Iddec2025;MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;
```

créer un « App Services » (« Web App »).

## Create Web App ...

### Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription \* ⓘ

Resource Group \* ⓘ  [Create new](#)

### Instance Details

Name \*  .azurewebsites.net

Publish \* ☒ Code ☐ Container ☐ Static Web App

Runtime stack \*

Operating System \* ☐ Linux ☒ Windows

Region \*

**i** Not finding your App Service Plan? Try a different region or select your App Service Environment.

### Pricing plans

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#) ⓘ

Windows Plan (Switzerland North) \* ⓘ  [Create new](#)

Pricing plan  [Explore pricing plans](#)

### Zone redundancy

An App Service plan can be deployed as a zone redundant service in the regions that support it. This is a deployment time only decision. You can't make an App Service plan zone redundant after it has been deployed. [Learn more](#) ⓘ

Zone redundancy ☐ Enabled: Your App Service plan and the apps in it will be zone redundant. The minimum App Service plan instance count will be three.

☒ Disabled: Your App Service Plan and the apps in it will not be zone redundant. The minimum App Service plan instance count will be one.

### Recommended services (preview)

Within a new App Service plan, you can deploy these commonly-used services with your app. Services you select will be deployed secure-by-default. [Learn More](#) ⓘ

**i** Additional services are not supported in a Free App Service plan.

[Review + create](#)

[< Previous](#)

[Next : Deployment >](#)

# Create Web App ...

Basics Deployment Networking Monitoring Tags Review + create

## Summary

**Web App**  
by Microsoft

**Free sku**  
Estimated price - Free

 Basic authentication for this app is currently disabled and may impact deployments. Click to learn more.

## Details

|                |                                      |
|----------------|--------------------------------------|
| Subscription   | 2c21f6ee-9263-469d-956a-9962f87b8b49 |
| Resource Group | YvesAEffacerPlusTard                 |
| Name           | idec2024                             |
| Publish        | Code                                 |
| Runtime stack  | .NET 8 (LTS)                         |

## App Service Plan (New)

|                  |                       |
|------------------|-----------------------|
| Name             | idec2024              |
| Operating System | Windows               |
| Region           | Switzerland North     |
| SKU              | Free                  |
| ACU              | Shared infrastructure |
| Memory           | 1 GB memory           |

## Monitoring

|                      |             |
|----------------------|-------------|
| Application Insights | Not enabled |
|----------------------|-------------|

## Deployment

|                       |                                         |
|-----------------------|-----------------------------------------|
| Basic authentication  | Disabled                                |
| Continuous deployment | Not enabled / Set up after app creation |

**YvesAEffacerPlusTard**   

  Create  Manage view  Delete resource group  Refresh  Export to CSV  Open query  Assign tags  Move  Delete  Export template  Open in mobile 

**Overview**  Activity log  Access control (IAM)  Tags  Resource visualizer  Events **Settings**  Deployments  Security  Deployment stacks  Policies  Properties  Locks

**Essentials**  Subscription [\(move\)](#) : [LoroGS - DevQA](#)  Deployments : [2 Succeeded](#)  
 Subscription ID : 2c21f6ee-9263-469d-956a-9962f87b8b49  Location : Switzerland North  
 Tags [\(edit\)](#) : [Add tags](#)

**Resources**  Recommendations  
  Type equals **all**  Location equals **all**  Add filter  
Showing 1 to 4 of 4 records. ☐ Show hidden types  No grouping  List view  

| <input type="checkbox"/> Name ↑↓                      | Type ↑↓          | Location ↑↓       |     |
|-------------------------------------------------------|------------------|-------------------|-----|
| <input type="checkbox"/> idec2024                     | SQL server       | Switzerland North | ... |
| <input type="checkbox"/> idec2024 (idec2024/idec2024) | SQL database     | Switzerland North | ... |
| <input type="checkbox"/> idec2024                     | App Service      | Switzerland North | ... |
| <input type="checkbox"/> idec2024                     | App Service plan | Switzerland North | ... |

## Localisation

Dans Program.cs ajouter les lignes

```
/*DEBUT CUSTOM*/  
CultureInfo cultureInfo = new CultureInfo("fr-CH");
```

```
cultureInfo.NumberFormat.NumberGroupSeparator = " ";  
cultureInfo.NumberFormat.CurrencyGroupSeparator = " ";  
cultureInfo.NumberFormat.NumberDecimalSeparator = ".";  
cultureInfo.NumberFormat.NumberDecimalDigits = 2;  
  
CultureInfo.DefaultThreadCurrentCulture = cultureInfo;  
CultureInfo.DefaultThreadCurrentUICulture = cultureInfo;  
/*FIN CUSTOM*/
```

## Problème certificat https

dotnet dev-certs https --clean

dotnet dev-certs https --trust

## TODO

Mettre la page de modification de l'utilisateur (sans mot de passe) et la page de modification du mot de passe